

1. Part 29 - Why not put everything in Program.cs?

- We separate the code so each class has one job. This makes the program easier to read, test, fix, and update. For example, Program.cs starts the application; ConsoleMenu handles the user input, and WeatherApiClient communicates with OpenWeather.

Part 30 - Design Questions

1. HTTP

What HTTP method did we use?

Why is GET appropriate for retrieving weather information?

- We used the GET method. It is appropriate because the application only asks OpenWeather for weather data. It does not create, edit, or delete data.

2. JSON

Why does the API return JSON instead of a C# object?

What process converts JSON into our C# DTO?

- The API returns JSON because JSON is a common format that can be used by many programming languages, not only C#. System.Text.Json converts the JSON response into C# DTO objects. In the program, ReadFromJsonAsync performs this conversion.

3. DTO

Why did we create:

CurrentWeatherDto

ForecastDto

ForecastItemDto

Instead of directly working with raw JSON?

- We use DTOs such as CurrentWeatherDto, ForecastDto, and ForecastItemDto to organize the API data into C# objects. This is easier and safer than manually reading raw JSON text. DTOs also let us use clear property names such as Temperature, Humidity, and Items.

4. Configuration

Why is this: "ApiKey": "..." stored in appsettings.json instead of: var apiKey = "..."; inside Program.cs?

- The API key is stored in appsettings.json so it is separate from the program code. This makes it easier to change the key, base URL, or units without editing Program.cs. It also helps prevent sensitive values from being mixed with application logic.

5. Error Handling

Explain how the application currently handles each situation.

What happens if:

1. The city does not exist?

- OpenWeather returns a 404 Not Found response. The API client returns null, and the menu displays: City not found.

2. The API key is invalid?

- OpenWeather returns an error, usually 401 Unauthorized. EnsureSuccessStatusCode() throws an exception, which is handled by the application's error handling.

3. There is no internet connection?

- The HTTP request fails and throws an exception. The application catches the error and shows that an unexpected error occurred.

4. The API is unavailable?

- The request may fail with an HTTP error or timeout. The application catches the exception instead of crashing suddenly.

5. The user presses Ctrl+C?

- The cancellation token is cancelled. The application shows Shutdown requested... and stops safely.

6. The API returns an unexpected response?

- JSON conversion may fail, or the request may throw an exception. The application catches the exception, logs it, and displays an error message.